

Project 1 Report

2022059952 박준서

Design



Design

Explain your implementation plan to meet the assignment requirements.

Project 1에서 해야 하는 일은 기존 xv6에서 Round-Robin 방식으로 구현된 스케줄러를 Completely Fair Scheduler로 바꾸는 것이다. CFS를 구현하기 위해서는 기본적으로 각 프로세스가 자신의 virtual runtime과 nice value를 가지고 있어야 하므로, 기존 xv6에서 정의된 `proc` 구조체에 virtual runtime을 나타내는 변수 `vruntime` 과 priority를 결정할 nice value `nice` 를 추가하기로 했다.

한편 CFS에서는 `RUNNABLE` 한 프로세스 중 가장 작은 `vruntime` 을 가진 것을 꺼내기 위해 Self-balancing tree인 Red-Black Tree를 이용한다. 과제 명세에서도 `rbtree.h` 와 `rbtree.c` 를 통해 구현된 Red-black Tree를 사용하도록 하고 있으므로, 기존 구현된 `rb_node` 구조체와 프로세스 `proc` 을 mapping하여, `vruntime` 을 기준으로 tree가 self-balancing을 수행하도록 해 "다음 스케줄링 대상"을 용이하게 알아낼 수 있도록 했다.

과제 명세에서는 정해진 nice value(`-3` 부터 `2` 까지의 정수)에 따라 프로세스가 CPU를 점유한 시간 대비 증가하는 `vruntime` 의 비를 제공하고 있다. 가장 작은 nice value(`-3`)을 가졌을 때 CPU 점유 시간당 `vruntime` 의 증가량을 `1` 로 하면 다음과 같이 nice value에 따른 CPU 점유 시간당 `vruntime` 증가량을 정리할 수 있다.

nice value	CPU 점유 시간당 vruntime 증가량
-3	1
-2	2
-1	4
0	8
1	16
2	32

`proc` 구조체에서 `vruntime` 변수를 `uint64` type으로 정의하면 매우 큰 수까지 `vruntime` 을 다룰 수 있으나, 혹여나 overflow가 일어날 가능성을 제거하기 위해 CPU 점유 시간당 `vruntime` 의 증가량을 최소 배수로 하여 구현하기로 하였다. `vruntime` 은 CPU를 점유하고 있던 프로세스가 Timer Interrupt 등의 이유로 CPU를 내려놓고 `RUNNING` 상태로 돌아올 때, 즉 코드 상으로는 `scheduler()` 함수의 내부로 돌아올 때 CPU 점유 시간에 비례하여 위 표에 맞춰 증가시키기로 하였다.

Implementation



Design

Detail the specific modifications and additions you made to the given xv6 codebase.

Clearly highlight how they differ from the original code and the purpose of these changes.

Step 1. `proc` 구조체 수정

```

85
86 #include "rbtree.h" // for define rb_node type member variable (procnode)
87
88 struct proc {
89     struct spinlock lock;
90
91     // p->lock must be held when using these:
92     enum procstate state; // Process state
93     void *chan; // If non-zero, sleeping on chan
94     int killed; // If non-zero, have been killed
95     int xstate; // Exit status to be returned to parent's wait
96     int pid; // Process ID
97
98     // wait_lock must be held when using this:
99     struct proc *parent; // Parent process
100
101     // these are private to the process, so p->lock need not be held.
102     uint64 kstack; // Virtual address of kernel stack
103     uint64 sz; // Size of process memory (bytes)
104     pagetable_t pagetable; // User page table
105     struct trapframe *trapframe; // data page for trampoline.S
106     struct context context; // switch() here to run process
107     struct file *ofile[NFILE]; // Open files
108     struct inode *cwd; // Current directory
109     char name[16]; // Process name (debugging)
110
111     // for project.
112     uint64 vruntime; // virtual runtime of this process
113     int nice; // nice value
114     struct rb_node procnode;
115 };
116

```

CFS를 구현하기 위해 각 프로세스가 Virtual Runtime과 Nice Value를 갖고, Virtual Runtime에 따라 self-balancing 하는 Red-black Tree와 one-to-one으로 mapping할 수 있도록 `vruntime`, `nice`, `procnode` 를 `proc` 구조체의 멤버 변수로 추가하였다.

Step 2. Ready Queue 정의

CFS를 구현할 때 `RUNNABLE` 한 프로세스가 대기할 Ready Queue는 xv6 운영체제에 기 구현된 `kernel/rbtree.h` 와 `kernel/rbtree.c` 에서 제공되는 Red-Black Tree를 이용해 구현하였다.

Multiprocessor 환경까지 고려하였을 때 각 CPU가 Ready Queue와 그 Ready Queue에 들어있는 `RUNNABLE` 한 프로세스에 접근할 수 있어야 하므로 Ready Queue는 `kernel/proc.c` 에 **global variable**로 선언하였고, 또한 서로 다른 CPU가 Ready Queue에 동시에 접근함으로써 발생할 위험이 있는 Race Condition을 차단하기 위해 Ready Queue에 대한 Lock을 `struct spinlock rbtree_lock;` 으로 구현하였다.

```

29 // For project.
30 // CFS Ready queue and lock for Ready queue.
31 struct spinlock rbtree_lock;
32 struct rb_tree ready_queue; // cfs ready queue
33

```

Step 3. `procinit()` 함수 수정

```

52 // initialize the proc table.
53 void
54 procinit(void)
55 {
56     struct proc *p;
57
58     ready_queue.root = 0; // initialize to NULL
59
60     initlock(&pid_lock, "nextpid");
61     initlock(&wait_lock, "wait_lock");
62     initlock(&rbtree_lock, "rbtree_lock");
63     for(p = proc; p < &proc[NPROC]; p++) {
64         initlock(&p->lock, "proc");
65         p->state = UNUSED;
66         p->kstack = KSTACK((int) (p - proc));
67     }
68 }

```

`procinit()` 함수는 운영체제를 부팅할 때 Process table을 만드는 역할을 한다. **Step 2**에서 Ready Queue와, 그 ready queue의 동기화 문제를 해결하기 위한 `rbtree_lock`을 선언하였으므로 이들을 초기화하는 로직을 추가하였다.

Step 4. 프로세스 생성 관련 함수 수정

→ `allocproc()`, `userinit()`, `kfork()`

기존 xv6 운영체제에서 새로운 프로세스는 `userinit()` 함수나 `kfork()` 함수를 통해 만들어진다.

`userinit()` 함수는 운영체제를 시작했을 때 첫 번째 프로세스(`init`)를 만들 때 호출되며, 부모 프로세스로부터 자식 프로세스를 복제할 때는 System Call `fork()`를 통해 `kfork()`를 간접적으로 호출한다.

이때 두 함수 `userinit()` 과 `kfork()`는 함수 내부에서 `allocproc()`을 호출함으로써, Process Table에서 UNUSED인 `proc`을 찾아 kernel이 실행할 수 있는 상태가 되도록 준비하는 작업을 거치고, 이후 만들어진 프로세스의 state를 `RUNNABLE`로 바꾸는 역할까지 수행한다.

우리는 Step 1에서 `proc` 구조체에 새로운 멤버 변수를 추가하였고, CFS에서 `RUNNABLE`한 프로세스를 Ready Queue에서 관리해야 하므로 각 함수에서 다음과 같은 과정을 추가하였다.

`allocproc()`

`proc` 구조체에 `vruntime`, `nice`, `procnode`를 추가하였으므로, 새로 할당한 프로세스에는 `vruntime = 0`, `nice = 0`으로 두었고, `procnode`의 각 멤버 변수는 다음과 같이 정의하였다.

이 함수가 `userinit()`이나 `kfork()`에서 호출된 후, 각 프로세스는 첫 번째 프로세스(`init`) 또는 자식 프로세스가 가져야 할 조건에 맞게 `vruntime`이나 `nice`를 갖게 될 것이다.

한편 Red-Black Tree와 관련 있는 필드 중 `parent`, `left`, `right`는 insert를 수행하기 전 모두 `NULL`로 설정하고, Red-Black Tree의 property에 따라 insert하는 node는 일단 `RED` color를 가지므로 다음과 같이 정의하였다.

```

131     return 0;
132
133     found:
134     p->pid = allocpid();
135     p->state = USED;
136
137     p->vruntime = 0;
138     p->nice = 0;
139     p->procnode.key = 0;
140     p->procnode.data = 0;
141     p->procnode.parent = 0;
142     p->procnode.left = 0;
143     p->procnode.right = 0;
144     p->procnode.color = RED;
145
146     // Allocate a trapframe page.
147     if((p->trapframe = (struct trapframe *)kalloc

```

`userinit()`

과제 명세에는 첫 번째 프로세스(the very first process)는 Virtual Runtime과 Nice Value를 0으로 설정하도록 하고 있다. 이에 따라 운영체제가 시작하고 첫 번째로 만들어지는 프로세스가 `vruntime = 0`과 `nice = 0`을 갖도록 했고, 이 프로세스와 mapping되는 `procnode`의 `key`를 `vruntime`으로, `data`를 이 프로세스 자체로 설정한 후 `rb_insert()` 함수를 통해 Ready Queue에 넣어주었다. 이때 Race Condition 방지를 위해 `rb_insert()`를 수행하기 직전 `rbtree_lock`을 acquire하였고, insert 직후 release하도록 하였다.

한편 `allocproc()` 함수에서 프로세스와 mapping되는 `procnode`의 `key`를 0으로 초기화하였기 때문에 `userinit()` 함수에서 Line 259와 같이 `p->procnode.key = p->vruntime;`을 굳이 수행할 필요는 없었지만, 다른 함수들과 패턴을 맞추기 위해 스케줄러 완성 후 해당 line을 작성해주었다.

또 특이할 점으로, `userinit()` 함수에서는 프로세스 `p`에 대한 lock을 명시적으로 acquire해주지 않고 있으나, `allocproc()` 함수에서 이미 `p`에 대한 lock을 acquire한 후 release하지 않은 상태에서 return을 하고 있으므로 이 함수 안에서 별도로 acquire를 해줄 필요가 없다.

```

245 // Set up first user process.
246 void
247 userinit(void)
248 {
249     struct proc *p;
250
251     p = allocproc();
252     initproc = p;
253
254     p->cwd = namei("/");
255
256     p->state = RUNNABLE;
257     p->vruntime = 0;
258     p->nice = 0;
259     p->procnode.key = p->vruntime;
260     p->procnode.data = (void*) p;
261     // NO additional operation about p->procnode.
262     // already initialized in allocproc() function.
263
264     acquire(&rbtree_lock);
265     rb_insert(tree, &ready_queue, node: &(p->procnode));
266     release(&rbtree_lock);
267
268     release(&p->lock);
269 }

```

kfork()

과제 명세에서는 `fork()` 를 통해 자식 프로세스가 만들어질 경우, 자식 프로세스는 `fork()` 시점의 부모 프로세스로부터 Virtual Runtime과 Nice Value를 상속받는다고 했다.

기존 xv6 코드의 `kfork()` 에서 `p` 는 부모 프로세스, `np` 는 새로 생성할 자식 프로세스(new process)에 대한 포인터를 나타낸다.

`userinit()` 함수에서와 유사하게, 새로 생성되는 자식프로세스 `np` 가 부모 프로세스 `p` 로부터 `vruntime` 과 `nice` 를 상속받도록 다음과 같은 로직을 추가하였다. (Line 336, Line 339)

한편 새로 생성된 자식 프로세스 `np` 도 Ready Queue에서 관리될 수 있도록 `procnode.key` 와 `procnode.data` 를 각각 자신의 `vruntime` , 그리고 자식 프로세스 자기 자신으로 두었고, (Line 338, Line 337)

이 `np->procnode` 를 Ready Queue에 insert해줌으로써 `RUNNABLE` 한 프로세스 `np` 를 Ready Queue에서 관리할 수 있도록 하였다. (Line 348)

한편 Line 334~342에서, 자식 프로세스 `np` 의 멤버 변수의 값을 부모 프로세스 `p` 로부터 가져오므로, 자식 프로세스 - 부모 프로세스 쌍 전체에 대한 lock을 통해 synchronization을 실현해줘야 한다. xv6에서는 자식 프로세스와 부모 프로세스 쌍을 한꺼번에 관리하는 `wait_lock` 을 별도로 정의하고 있고, 기존 `kfork()` 함수에서도 `np->parent = p;` 를 해당 `wait_lock` 이 acquire되어 있는 block에서 하고 있으므로 `vruntime` 과 `nice` 의 값을 상속받는 과정도 이 block 안에서 구현하였다.

```

kernel > C proc.c > kfork
297   kfork(void)

334   acquire(&wait_lock);
335   np->parent = p;
336   np->vruntime = p->vruntime;
337   np->procnode.data = (void*) np;
338   np->procnode.key = np->vruntime;
339   np->nice = p->nice;
340   // NO additional operation about np->procnode.
341   // already initialized in allocproc() function.
342   release(&wait_lock);
343
344   acquire(&np->lock);
345   np->state = RUNNABLE;
346
347   acquire(&rbtree_lock);
348   rb_insert(tree: &ready_queue, node: &np->procnode);
349   release(&rbtree_lock);
350
351   release(&np->lock);
352
353   return pid;
354 }

```

Step 5. 프로세스 상태변경 관련 함수 수정

→ `wakeup()`, `kill()`

xv6에서 `RUNNING` 상태에 있는 프로세스는 interrupt를 받거나, 자신의 자식 프로세스를 fork한 후 `wait()`를 호출하여 자식 프로세스가 죽을 때까지 기다릴 때 `SLEEPING` 상태로 전환되며 CPU를 내려놓는다.

이 `SLEEPING` 프로세스가 `RUNNABLE`로 바뀌는 경로는 두 가지로, 각각 `wakeup()` 함수와 `kill()` 함수가 호출되었을 때이다.

`wakeup()`

일반적으로 CFS에서, `SLEEPING` 중이었던 프로세스는 `RUNNABLE`로 상태가 변화했을 때 다른 프로세스와의 CPU 점유 공정성을 위해 `vruntime`을 업데이트해야 한다고 한다. 과제 명세에서는 프로세스가 깨어날 때 Ready Queue에 `RUNNABLE` 프로세스가 존재하는지, 그렇지 않다면 CPU에 `RUNNING` 중인 프로세스가 있는지를 확인하여 깨어난 프로세스의 `vruntime`을 업데이트하도록 하고 있다.

- When a process wakes up from `SLEEPING`, update its virtual runtime according to the following rules, evaluated in order:
 - If there are any `RUNNABLE` processes, set the waking process's virtual runtime to the minimum virtual runtime among all `RUNNABLE` processes.
 - If there are no `RUNNABLE` processes, but at least one process is currently `RUNNING`, set the waking process's virtual runtime to the lowest virtual runtime of any `RUNNING` process.
 - If there are neither `RUNNABLE` nor `RUNNING` processes, set the waking process's virtual runtime to 0.

Project 1 Specification.

수정한 `wakeup()` 함수에서는 깨어난 프로세스의 virtual runtime을 `new_vruntime`이라는 새로운 변수를 통해 업데이트할 수 있도록 하였다. Rule 1부터 Rule 3까지를 차례대로 따져가며 Ready Queue에 접근하므로, Rule 10에 대해 따지기 전

`rbtree_lock` 을 acquire함으로써 Ready Queue에 대한 Race Condition의 가능성을 차단하였다.

```
679         if(p->state == SLEEPING && p->chan == chan) {
680             p->state = RUNNABLE;
681
682             uint64 new_vruntime = 0;
683
684             acquire(&rbtree_lock);
685
```

Rule 1.

`RUNNABLE` 한 프로세스가 존재한다는 것은 그 시점에서 Ready Queue가 비어있지 않다는 것을 의미한다. 따라서 `rb_first()` 를 통해 `rb_node* n` 을 얻은 후, 이 `n` 이 `NULL` 이 아닌 경우를 **Rule 1**에 해당하는 경우라고 판단하여 그 `rb_node* n` 에 mapping되는 프로세스의 `vruntime` 을 가져와 `new_vruntime` 에 넣도록 하였다. 이 프로세스가 `RUNNABLE` 한 프로세스 중 `vruntime` 이 가장 작은 것이기 때문이다.

```
686         // Rule 1.
687         // If there are any RUNNABLE Processes,
688         // set the waking process's vruntime to the minimum vruntime
689         // among all RUNNABLE processes.
690         struct rb_node* n = rb_first(node; ready_queue.root);
691         if(n != 0){ // if RUNNABLE process exists in Ready Queue
692             new_vruntime = ((struct proc*)(n->data))->vruntime;
693         }
```

Rule 2.

한편 `rb_node* n` 이 `NULL` 인 경우는 Ready Queue에 어떤 노드도 들어있지 않음을 의미한다. 이 경우에는 모든 CPU를 순회하며 `RUNNING` 상태인 프로세스를 확인해, 그 시점에서 가장 `vruntime` 이 작은 프로세스를 가져오기로 하였다.

`kernel/proc.h` 에 정의된 `struct cpu` 에는 그 cpu에서 “실행 중인” 프로세스를 가리키는 포인터 `proc` 이 멤버 변수로 있으므로, 이 멤버 변수를 이용해 각 CPU에서 실행 중인 프로세스가 존재하는지 확인한 후 그 프로세스의 `vruntime` 을 확인하도록 하였다.

Ready Queue에 어떤 노드도 들어있지 않음을 확인했을 때, `min_vruntime` 이라는 새로운 변수를 `uint64` 가 허용하는 가장 큰 값 `~0ULL` (Bitwise NOT 이용)으로 초기화한 다음, 각 CPU를 순회하며 CPU 안에서 실행 중인 프로세스의 `vruntime` 이 `min_vruntime` 보다 작을 때마다 `min_vruntime` 을 최신화하는 방식으로 서로 다른 CPU들 사이에서 `vruntime` 의 최솟값을 구하였다.

```
695         // Rule 2.
696         // If there are no RUNNABLE processes, but at least one process is currently RUNNING,
697         // set the waking process's virtual runtime to the lowest virtual runtime
698         // of any RUNNING processes.
699         else{ // n == 0
700             uint64 min_vruntime = ~0ULL; // initial to MAX
701             for(struct cpu* c = cpus; c < &cpus[NCPU]; c++){
702                 if(c->proc != 0 && c->proc->state == RUNNING){ // if c runs process
703                     if(min_vruntime > c->proc->vruntime){
704                         min_vruntime = c->proc->vruntime;
705                     }
706                 }
707             }
```

Rule 3.

Rule 2에서 모든 CPU를 확인했음에도 불구하고 `RUNNING` 중인 프로세스를 단 하나도 찾지 못한 경우에는 for문을 완료하더라도 `min_vruntime` 이 `~0ULL` 일 것이다. 이 경우에는 `min_vruntime` 을 0으로 설정한 후 assign함으로써 `new_vruntime` 이 0을 갖도록 하였다.

```

708
709 // Rule 3.
710 // If there are neither RUNNABLE nor RUNNING processes,
711 // set the waking process's virtual runtime to 0.
712 if(min_vruntime == ~0ULL) min_vruntime = 0;
713 new_vruntime = min_vruntime;
714 }
715
716 p->vruntime = new_vruntime;
717 p->procnode.key = p->vruntime;
718
719 rb_insert(tree; &ready_queue, node: &p->procnode);
720
721 release(&rbtree_lock);
722 }
723
724 release(&p->lock);
725 }
726 }

```

kill()

SLEEPING 중인 프로세스를 죽여야 하는 경우에도 xv6는 먼저 해당 프로세스를 **RUNNABLE** 상태로 만든 후, 그 프로세스가 CPU를 잡았을 때 **kill()** 에서 set한 **p->killed** 필드를 확인해 **exit()** 하도록 하므로 **wakeup()** 과 동일한 로직을 갖도록 하였다.

```

731 kkill(int pid)
732 {
733     struct proc *p;
734
735     for(p = proc; p < &proc[NPROC]; p++){
736         acquire(&p->lock);
737         if(p->pid == pid){
738             p->killed = 1;
739             if(p->state == SLEEPING){
740                 p->state = RUNNABLE;
741
742                 uint64 new_vruntime = 0;
743
744                 acquire(&rbtree_lock);
745                 |
746                 // Rule 1.
747                 // If there are any RUNNABLE Processes,
748                 // set the waking process's vruntime to the minimum vruntime
749                 // among all RUNNABLE processes.
750                 struct rb_node* n = rb_first(node: ready_queue.root);
751                 if(n != 0){ // if RUNNABLE process exists in Ready Queue
752                     new_vruntime = ((struct proc*)(n->data))->vruntime;
753                 }
754
755                 // Rule 2.
756                 // If there are no RUNNABLE processes, but at least one process is currently RUNNING,
757                 // set the waking process's virtual runtime to the lowest virtual runtime
758                 // of any RUNNING processes.
759                 else{ // n == 0
760                     uint64 min_vruntime = ~0ULL; // initial to MAX
761                     for(struct cpu* c = cpus; c < &cpus[NCPU]; c++){
762                         if(c->proc != 0 && c->proc->state == RUNNING){ // if c runs process
763                             if(min_vruntime > c->proc->vruntime){
764                                 min_vruntime = c->proc->vruntime;
765                             }
766                         }
767                     }
768                 // Rule 3.
769                 // If there are neither RUNNABLE nor RUNNING processes,
770                 // set the waking process's virtual runtime to 0.
771                 if(min_vruntime == ~0ULL) min_vruntime = 0;
772                 new_vruntime = min_vruntime;
773             }
774
775             p->vruntime = new_vruntime;
776             p->procnode.key = p->vruntime;
777
778             rb_insert(tree: &ready_queue, node: &p->procnode);
779
780             release(&rbtree_lock);
781         }
782         release(&p->lock);
783         return 0;
784     }
785 }

```

Step 6. **scheduler()** 함수 수정

앞선 Step 2~4에서 프로세스를 관리할 Ready Queue와, 프로세스의 생성/상태 변경에서 고려해야 할 사항들을 반영하였으므로, 이 Ready Queue를 이용해 `scheduler()` 함수가 CFS를 잘 구현할 수 있도록 수정하였다.

먼저 CFS Scheduling은 `scheduler()` 함수 내부에서 다음과 같은 과정을 거쳐 수행된다.

1. Ready Queue에 대한 lock `rbtree_lock`을 acquire한 후, Ready Queue에서 가장 작은 `vruntime`을 가진 노드를 찾는다.
 - a. 이 노드에 대한 포인터 `n`이 `NULL`이 아니면(`n != 0`), Ready Queue에 `RUNNABLE`한 프로세스가 존재한다는 뜻이므로 이 노드에 mapping되는 프로세스를 다음 스케줄링의 대상으로 선택한다.
 - b. 만약 `NULL`이라면, `RUNNABLE`한 프로세스가 없다는 뜻이므로 이 CPU는 수행할 일이 없다. Interrupt가 발생할 때까지 대기한다.
2. 1-a에서 다음 스케줄링의 대상이 될 프로세스를 선택하면, Context Switch를 수행하기 전 해당 프로세스에 대한 노드를 Ready Queue에서 delete한다.
3. Context Switch를 **수행할** 프로세스 `p`에 대한 lock을 acquire하고, Context Switch가 일어나기 직전 timer의 값을 `start_tick` 변수를 통해 스탬핑한다.
4. Context Switch를 통해 **돌아온** 프로세스 `p`에 대하여 `delta_tick = ticks - start_tick;`을 통해 프로세스가 CPU를 실제로 몇 tick 동안 점유했는지를 저장한다. 만약 1 tick보다 짧은 시간 동안만 CPU를 점유하고 있었다면 `delta_tick`은 1로 둔다.
5. 프로세스가 CPU를 점유하고 있던 시간 `delta_tick`과 이 프로세스의 nice value에 따라 Design에서 설계한 대로 프로세스의 `vruntime`을 증가시킨 후, 이 프로세스와 mapping되는 노드의 `key` 값도 새로 계산한 `vruntime`의 값으로 업데이트한다.
6. CPU를 내려놓은 프로세스는 Timer Interrupt에 의해 내려놓았을 수도 있고, 자식 프로세스 fork 후 `wait()`를 호출하여 `SLEEPING` 상태에 빠진 것일 수도 있다. 프로세스가 `RUNNABLE`한 상태여야만 Ready Queue에 들어갈 수 있으므로 `p->state == RUNNABLE`인 조건문 안에서 해당 프로세스에 mapping되는 노드를 Ready Queue에 insert한다.

이 과정을 다음과 같이 `scheduler()` 함수 안에서 구현하였다.

```

472 void
473 scheduler(void)
474 {
475     struct proc *p;
476     struct cpu *c = mycpu();
477
478     c->proc = 0;
479     p = 0;
480     for(;;){
481         // The most recent process to run may have had interrupts
482         // turned off; enable them to avoid a deadlock if all
483         // processes are waiting. Then turn them back off
484         // to avoid a possible race between an interrupt
485         // and wfi.
486         intr_on();
487         intr_off();
488
489         int found = 0; // found process?
490         // dequeue
491         acquire(&rbtree_lock);
492
493         struct rb_node* n = rb_first(node: ready_queue.root);
494         if(n != 0){ // there exists RUNNABLE process in ready queue
495             p = (struct proc*)(n->data);
496             rb_delete(tree: &ready_queue, node: n);
497         }
498         release(&rbtree_lock);

```

`rbtree_lock`을 acquire한 후 Ready Queue에서 `vruntime`이 가장 작은 노드를 가져온다. `NULL`이 아닌 경우만 Ready Queue에 프로세스가 존재했다는 뜻이므로 해당 프로세스를 다음 스케줄링의 대상으로 선택한다.

```
// ready for switch context!
if(p != 0){
    acquire(&p->lock);
    if(p->state == RUNNABLE){
        p->state = RUNNING;
        c->proc = p;

        found = 1;
        uint64 start_tick = ticks;

        switch(&c->context, &p->context); // context switch!!

        uint64 delta_tick = ticks - start_tick;
        if(delta_tick == 0) delta_tick = 1;
    }
}
```

해당 프로세스에 대한 lock을 acquire하고 상태를 RUNNABLE로 바꾼 다음 context switch를 진행한다. switch() 함수 전후로 그 시점의 tick을 잴으로써 프로세스의 CPU 점유 시간을 측정한다.

```
switch(p->nice){
    case -3:
        p->vruntime += delta_tick;
        break;
    case -2:
        p->vruntime += delta_tick * 2;
        break;
    case -1:
        p->vruntime += delta_tick * 4;
        break;
    case 0:
        p->vruntime += delta_tick * 8;
        break;
    case 1:
        p->vruntime += delta_tick * 16;
        break;
    case 2:
        p->vruntime += delta_tick * 32;
        break;
    default:
        panic("Invalid nice value\n");
}

p->procnode.key = p->vruntime;

if(p->state == RUNNABLE){
    acquire(&rbtree_lock);
    rb_insert(tree, &ready_queue, node, &p->procnode);
    release(&rbtree_lock);
}
```

CPU 점유 시간(delta_tick)과 프로세스의 nice value에 따라 vruntime을 업데이트하고, Timer Interrupt에 의해 CPU를 내려놓은 경우에만 (p->state == RUNNABLE) Ready Queue에 다시 insert해준다.

Step 7. System Call `set_nice()` 등록

과제 명세에 따라, 생성할 프로세스의 nice value를 세팅해주는 `set_nice()` System Call을 등록하였다.

nice value는 -3부터 2까지의 정수만 가능하므로, `kernel/proc.c`에서 이 범위의 정수가 argument `nice`로 들어오면 해당 프로세스의 nice value를 `nice`로 바꾸고 0을 return하고, 범위 밖의 정수가 들어온 경우 1을 return하도록 하였다.

한편 `set_nice()` 에서도, 프로세스의 nice value를 바꾸는 로직이 있으므로 이 critical section의 앞뒤로 lock을 acquire / release하도록 하였다.

```
869 // For Project.
870 int set_nice(int nice){
871     struct proc* p = myproc();
872     if(nice >= -3 && nice <= 2){
873         acquire(&p->lock);
874         p->nice = nice;
875         release(&p->lock);
876         return 0;
877     }
878     else{
879         return 1; // Invalid nice value
880     }
881 }
```

이후에는 지난 Assignment에서와 같이, `kernel/defs.h`, `kernel/sysproc.c`, `kernel/syscall.h`, `kernel/syscall.c`, `user/user.h`, `user/usys.pl` 에 해당 System Call에 대한 정보를 추가하여 User Program에서 `set_nice()` System Call을 호출할 수 있도록 했다.

```
102 void yield(void);
103 int either_copyout(int user_dst, uint64 dst, void *src, uint64 len);
104 int either_copyin(void *dst, int user_src, uint64 src, uint64 len);
105 void procdump(void);
106 int set_nice(int); // add new system call
107
```

kernel/defs.h에 함수 signature 추가.

```
110
111 uint64 sys_set_nice(void){
112     int nice;
113
114     argint(0, &nice);
115     return set_nice(nice);
116 }
```

kernel/sysproc.c에 Wrapper Function 추가.

```

22 #define SYS_close 21
23 #define SYS_set_nice 22
24

```

kernel/syscall.h에 System Call Number 추가.

```

104 extern uint64 sys_set_nice(void);
105
106 // An array mapping syscall numbers from
107 // to the function that handles the sys
108 static uint64 (*syscalls[])(void) = {
109     [SYS_fork]    sys_fork,
110     [SYS_exit]    sys_exit,
111     [SYS_wait]    sys_wait,
112     [SYS_pipe]    sys_pipe,
113     [SYS_read]    sys_read,
114     [SYS_kill]    sys_kill,
115     [SYS_exec]    sys_exec,
116     [SYS_fstat]   sys_fstat,
117     [SYS_chdir]   sys_chdir,
118     [SYS_dup]     sys_dup,
119     [SYS_getpid]  sys_getpid,
120     [SYS_sbrk]    sys_sbrk,
121     [SYS_pause]   sys_pause,
122     [SYS_uptime]  sys_uptime,
123     [SYS_open]    sys_open,
124     [SYS_write]   sys_write,
125     [SYS_mknod]   sys_mknod,
126     [SYS_unlink]  sys_unlink,
127     [SYS_link]    sys_link,
128     [SYS_mkdir]   sys_mkdir,
129     [SYS_close]   sys_close,
130     [SYS_set_nice] sys_set_nice,

```

kernel/syscall.c에서 Syscall Table에 추가.

```

26 int uptime(void);
27 int set_nice(int);
28

```

user/user.h에 set_nice() 함수 signature를 추가함으로써 User program에서 이 함수를 호출할 수 있도록 함.

```

42 entry("sbrk");
43 entry("pause");
44 entry("uptime");
45 entry("set_nice");
46

```

user/usys.pl에 entry를 추가하여 set_nice 함수를 호출하는 Assembly code를 만들 수 있도록 함.

Results



Results

Document the compilation and execution process. Include screenshots verifying that all requirements are met, alongside explanations of the execution flow.

Github Discussion에 올라온 다음 test를 이용해 CFS Scheduler execution을 확인하겠다. 각 프로세스가 Virtual Runtime에 따라 스케줄링이 되는지를 더 가시적으로 확인하기 위해 이 테스트에서 `scheduler()` 함수에 다음을 추가하였다.

```
// ready for switch context!
if(p != 0){
    acquire(&p->lock);
    if(p->state == RUNNABLE){

        // for debug.
        printf("Process selected : PID = %d, vruntime = %lu\n", p->pid, p->vruntime);

        p->state = RUNNING;
        c->proc = p;
    }
}
```

[Project 1] CFS 간단한 테스트 프로그램입니다 #28

YOrFa1se started this conversation in test-cases


YOrFa1se last week
edited ...


1. 테스트 목적 (Objective)

생성 순서와 상관없이 nice 값이 작은 프로세스가 먼저 종료되는지를 테스트합니다

잘못된 부분 있으면 댓글 부탁드립니다


2. 테스트 코드 (Test Code)

```
#include "kernel/types.h"
#include "user.h"

int main(int argc, char *argv[]){
    for (int nice = 2; nice >= -3; nice--){
        int pid = fork();
        if (pid < 0){
            printf("fork failed\n");
            exit(1);
        }

        if (pid == 0){
            set_nice(nice);
            for (volatile int i = 0; i < 100000000; i++){
                printf("pid: %d, nice: %d\n", getpid(), nice);
            }
            exit(0);
        }

        for (int i = 0; i < 6; i++){
            wait(0);
        }
    }

    return 0;
}
```

위 테스트는 nice value가 서로 다른 6개의 프로세스를 `fork()` 하였을 때 nice value가 작은 프로세스부터 종료되는지를 확인하는 테스트다. 실행 결과는 다음과 같고, "다음 스케줄링 대상이 된 프로세스"에 대한 정보를 보면 `vruntime` 이 가장 작은 것이 계속해서 출력되고 있다는 걸 확인할 수 있다.

```

nicetest1
Process wakeup : PID = 2, vruntime = 0
Process selected : PID = 2, vruntime = 0
Process selected : PID = 3, vruntime = 0
Process wakeup : PID = 3, vruntime = 0
Process selected : PID = 3, vruntime = 0
Process wakeup : PID = 3, vruntime = 0
Process selected : PID = 3, vruntime = 0
Process wakeup : PID = 3, vruntime = 0
Process selected : PID = 3, vruntime = 0
Process selected : PID = 4, vruntime = 0
Process selected : PID = 5, vruntime = 0
Process selected : PID = 6, vruntime = 0
Process selected : PID = 7, vruntime = 0
Process selected : PID = 8, vruntime = 0
Process selected : PID = 9, vruntime = 0
Process selected : PID = 9, vruntime = 1
Process selected : PID = 8, vruntime = 2
Process selected : PID = 9, vruntime = 2
Process selected : PID = 9, vruntime = 3
Process selected : PID = 7, vruntime = 4
Process selected : PID = 8, vruntime = 4
Process selected : PID = 9, vruntime = 4
pid: 9, nice: -3
Process wakeup : PID = 3, vruntime = 6
Process selected : PID = 8, vruntime = 6
Process selected : PID = 3, vruntime = 6
Process selected : PID = 6, vruntime = 8
Process selected : PID = 7, vruntime = 8
Process selected : PID = 8, vruntime = 8
pid: 8, nice: -2
Process wakeup : PID = 3, vruntime = 12
Process selected : PID = 7, vruntime = 12
Process selected : PID = 3, vruntime = 12
Process selected : PID = 5, vruntime = 16
Process selected : PID = 6, vruntime = 16
Process selected : PID = 7, vruntime = 16
pid: 7, nice: -1
Process wakeup : PID = 3, vruntime = 24
Process selected : PID = 6, vruntime = 24
Process selected : PID = 3, vruntime = 24
Process selected : PID = 4, vruntime = 32
Process selected : PID = 5, vruntime = 32
Process selected : PID = 6, vruntime = 32
pid: 6, nice: 0
Process wakeup : PID = 3, vruntime = 48
Process selected : PID = 5, vruntime = 48
Process selected : PID = 3, vruntime = 48
Process selected : PID = 4, vruntime = 64
Process selected : PID = 5, vruntime = 64
pid: 5, nice: 1
Process wakeup : PID = 3, vruntime = 96
Process selected : PID = 4, vruntime = 96
Process selected : PID = 3, vruntime = 96
Process selected : PID = 4, vruntime = 128
Process selected : PID = 4, vruntime = 160
pid: 4, nicProcess selected : PID = 4, vruntime = 192
e: 2
Process wakeup : PID = 3, vruntime = 192
Process selected : PID = 3, vruntime = 192
Process wakeup : PID = 2, vruntime = 192
Process selected : PID = 2, vruntime = 192
$Process wakeup : PID = 2, vruntime = 0
Process selected : PID = 2, vruntime = 0
□

```

이 execution에 대한 control flow를 따라가보면 다음과 같다.

1st. 운영체제를 실행하면 `init` (PID = 1)과 `sh` (PID = 2) 프로세스가 실행된다.

2nd. shell에 `nicetest1` 를 입력하면 `user.h` 에 등록된 `nicetest1` 이 PID = 3인 프로세스로 실행되며, 이 프로세스는 PID = 4 (nice == 2)인 프로세스부터 PID = 9인 (nice == -3)인 프로세스까지 순서대로 `fork()` 한 후 `SLEEPING` 상태가 된다.

3rd. active한 6개의 프로세스 PID = 4부터 PID = 9는 `scheduler()` 함수에서 구현한 CFS 정책에 따라 스케줄링되며 `vruntime` 을 증가시킨다.

4th. 동일한 코드를 수행하는 6개의 자식 프로세스 중 더 먼저 자주 실행되는 프로세스부터 종료되고 `ZOMBIE` 상태가 된다. `ZOMBIE` 가 된 자식 프로세스가 생겨나면 부모 프로세스(PID = 3)은 잠시 wakeup하여 자식을 `exit()` 시킨다.

5th. 6개의 자식 프로세스가 모두 종료되면 부모 프로세스도 종료된다.

Troubleshooting



Troubleshooting

If you encountered any issues during the assignment, describe the issues and your resolution process. For unresolved issues, explain what they were and what approaches you attempted.

Process - Node Mapping Issue (Resolved)

CFS Scheduler를 처음에 만들었을 때 OS는 실행되지만 실제 vruntime에 따른 우선순위가 반영되지 않고 스케줄링되는 문제가 있었다. 앞선 Result에서와 동일한 usertest를 진행했을 때 프로세스가 nice value의 오름차순대로 출력되지 않아, scheduler() 함수에서 다음 프로세스가 선택되었을 때 그 프로세스의 정보를 출력할 수 있도록 한 후 프로그램을 실행해보았다.

```
// ...

if(p != 0){
    acquire(&p->lock);
    if(p->state == RUNNABLE){

        // for debug.
        printf("Process selected : PID = %d, vruntime = %lu\n",
            p->pid, p->vruntime);

        // ...
    }
}
```

```

nicetest1
Schedule : PID 2, vruntime 0d
Schedule : PID 3, vruntime 0d
Schedule : PID 3, vruntime 0d
Schedule : PID 3, vruntime 0d
Schedule : PID 3, vruntime 0d
Schedule : PID 3, vruntime 0d
Schedule : PID 4, vruntime 0d
Schedule : PID 5, vruntime 0d
Schedule : PID 6, vruntime 0d
Schedule : PID 7, vruntime 0d
Schedule : PID 8, vruntime 0d
Schedule : PID 9, vruntime 0d
Schedule : PID 4, vruntime 32d
Schedule : PID 5, vruntime 16d
Schedule : PID 6, vruntime 8d
Schedule : PID 7, vruntime 4d
Schedule : PID 8, vruntime 2d
Schedule : PID 9, vruntime 1d
Schedule : PID 4, vruntime 64d
Schedule : PID 5, vruntime 32d
Schedule : PID 6, vruntime 16d
Schedule : PID 7, vruntime 8d
Schedule : PID 8, vruntime 4d
Schedule : PID 9, vruntime 2d
Schedule : PID 4, vruntime 96d
Schedule : PID 5, vruntime 48d
Schedule : PID 6, vruntime 24d
Schedule : PID 7, vruntime 12d
Schedule : PID 8, vruntime 6d
Schedule : PID 9, vruntime 3d
Schedule : PID 4, vruntime 128d
Schedule : PID 5, vruntime 64d
Schedule : PID 6, vruntime 32d
Schedule : PID 7, vruntime 16d
Schedule : PID 8, vruntime 8d
Schedule : PID 9, vruntime 4d
Schedule : PID 4, vruntime 160d
Schedule : PID 5, vruntime 80d
Schedule : PID 6, vruntime 40d
Schedule : PID 7, vruntime 20d
Schedule : PID 8, vruntime 10d
Schedule : PID 9, vruntime 5d
Schedule : PID 4, vruntime 192d
Schedule : PID 5, vruntime 96d
Schedule : PID 6, vruntime 48d
Schedule : PID 7, vruntime 24d
Schedule : PID 8, vruntime 12d
Schedule : PID 9, vruntime 6d
Schedule : PID 4, vruntime 224d
Schedule : PID 5, vruntime 112d
Schedule : PID 6, vruntime 56d
Schedule : PID 7, vruntime 28d
Schedule : PID 8, vruntime 14d
Schedule : PID 9, vruntime 7d
Schedule : PID 4, vruntime 256d
Schedule : PID 5, vruntime 128d
Schedule : PID 6, vruntime 64d
Schedule : PID 7, vruntime 32d
Schedule : PID 8, vruntime 16d
Schedule : PID 9, vruntime 8d

```

처음 프로그램을 실행했을 때 실행 결과.

vruntime은 규칙에 맞게 증가하지만 선택되는 프로세스가 RR 방식과 같다.

원인은 `scheduler()` 함수에서 `swtch()` 를 통해 CPU를 내려놓고 돌아온 프로세스가 `vruntime` 을 증가시킨 후 업데이트된 정보를 자신의 `procnode` 에 반영하지 않은 상태로 Ready Queue에 insert한 것이었다. 즉 `vruntime` 을 업데이트하는 로직 이후 `p->procnode.key = p->vruntime` 을 해주지 않아, 프로세스의 `vruntime` 은 증가했음에도 불구하고 그에 mapping되는 node의 `key` 는 그 값을 반영하지 못해 계속해서 `vruntime = 0`임을 대변한 상태로 Ready Queue에 들어가고 있었던 것이다.

이 때문에 `proc` 구조체 `p` 의 `vruntime` 은 실제로 증가하고 있었으나, Ready Queue에서는 모든 프로세스가 `vruntime = 0` 이다 보니 스케줄러가 Round-Robin과 같은 방식으로 동작하고 있었다.

이에 프로세스 `p` 와, 이 프로세스를 mapping하는 node의 `key` 까지 업데이트한 후 Ready Queue에 insert했더니 `vruntime` 이 가장 작은 `RUNNABLE` 프로세스를 다음 스케줄링의 대상으로 정상적으로 뽑을 수 있게 동작하였다.


```

869 // For Project.
870 int set_nice(int nice){
871     struct proc* p = myproc();
872     if(nice >= -3 && nice <= 2){
873         acquire(&p->lock);
874         p->nice = nice;
875         release(&p->lock);
876         return 0;
877     }
878     else{
879         return 1; // Invalid nice value
880     }
881 }

```